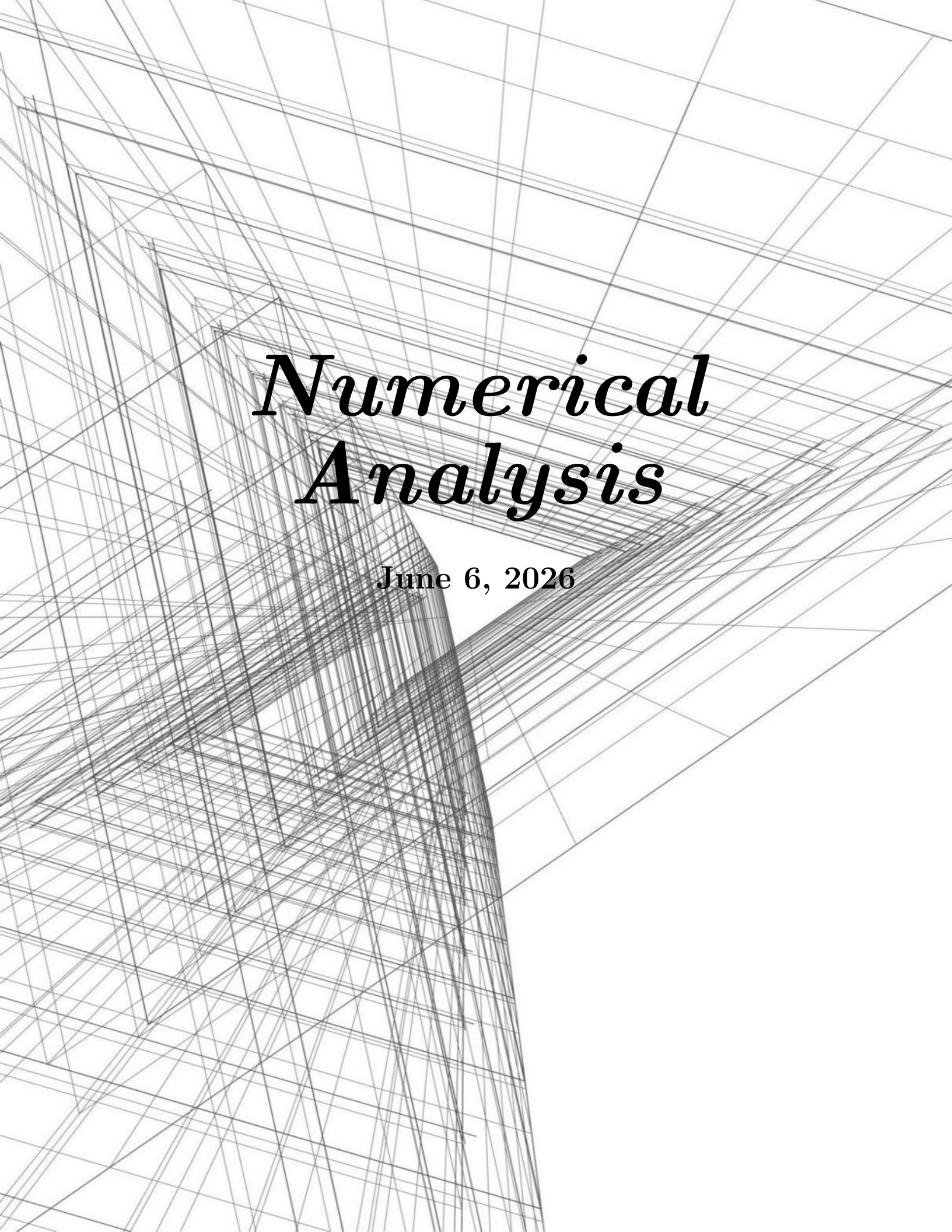

Numerical Analysis

June 6, 2026

Contents

Chapter 1

Preliminaries

1.1 Error Analysis

The arithmetic performed by a calculator or computer is not exact. In the computational world, each representable number has only a fixed and finite number of digits. This means, for example, that only some of the rational numbers can be represented exactly.

The error that is produced when a calculator or computer is used to perform real number calculations is called **round-off error**. In a computer, only a relatively small subset of the real number system is used for the representation of all the real numbers. This subset contains only rational numbers, both positive and negative, and stores the fractional part, together with an exponential part.

1.1.1 Binary Machine Numbers

IEEE 754-2008 provides standards for binary and decimal floating point numbers, formats for data interchange, algorithms for rounding arithmetic operations, and the handling of exceptions. Formats are specified for single, double, and extended precisions, and these standards are widely used in computer hardware and software.

A 64-bit (binary digit) representation is used for a real number. The first bit s is the sign bit, the next 11 bits are used for the exponent c, called the **characteristic**, and the remaining 52 bits are used for the fraction f, called the **mantissa**. To save storage and provide a unique representation for each floating-point number, a normalization is imposed.

A number with $c \neq 0$ is called a **normalized number**, and is represented as

$$N = (-1)^s 2^{c-1023}(1+f), \tag{1.1}$$

This represents a number in the range

$$[N - 2^{-52}, N + 2^{-52})$$

Numbers occurring in calculations that have a magnitude less than $2^{-1022}(1+0)$ results in a **underflow**, and is generally set to zero. Numbers with a magnitude greater than $2^{1023}(2-2^{-52})$ results in an **overflow**, and generally causes an error message to be generated.

When $c = 0$ and $f \neq 0$, the number is called a **denormalized number**, and is represented as

$$N = (-1)^s 2^{-1022} f. \tag{1.2}$$

zero is represented by $s = 0, 1, c = 0, f = 0$, and is called a **signed zero**.

1.1.2 Decimal Machine Numbers

The use of binary digits imposes difficulties to examine the problems from finite digit representation. So we shall use a normalized decimal representation of a real number, with the form

$$N = \pm 0.d_1 d_2 \cdots d_k \times 10^n, \qquad 1 \leqslant d_1 \leqslant 9, \quad 0 \leqslant d_i \leqslant 9, i = 2, 3, \cdots, k,$$

This number is called k-digit decimal machine number.

From analysis on the real line, there is a way to normalize any positive real number to the form

$$y = 0.d_1 d_2 \cdots d_k \cdots \times 10^n,$$

The floating point representation of y is obtained by terminating the mantissa after k digits, and is denoted by $fl(y)$. There are two common methods for terminating the mantissa:

- **Chopping**: simply drop off the digits after the k-th digit.
- **Rounding**: if $d_{k+1} \geqslant 5$, then add 10^{-k} to the chopped number; otherwise, simply chop off the digits after the k-th digit.

The error caused by replacing y by $fl(y)$ is called the **round-off error**. There are mainly two origins of errors:

- **Original error**: the error caused by the initial approximation of a number.
- **Round-off error**: the error caused by the finite digit representation of a number. (Rounding error and Chopping error are two types of round-off errors.)

Definition 1.1.1: Error

Suppose p^* is an approximation to a number p. The **actual error** is defined as $p - p^*$. The **absolute error** is defined as $|p - p^*|$. The **relative error** is defined as $\dfrac{|p - p^*|}{|p|}$, provided $p \neq 0$.
An error bound is a nonnegative number larger than the absolute error.

Remark:
We often cannot find an accurate value for the true error in an approximation. Instead, we find a bound for the error, which gives us a "worst-case" error.

Definition 1.1.2: Significant Digits

The number p^* is said to approximate p to t **significant digits**, if t is the largest nonnegative integer such that

$$\frac{|p - p^*|}{|p|} \leqslant 5 \times 10^{-t}. \tag{1.3}$$

The floating point representation of a number y has relative error $\dfrac{|y - fl(y)|}{|y|}$. If k-decimal chopping is used, then

$$\frac{|y - fl(y)|}{|y|} = \left| \frac{0.d_{k+1} d_{k+2} \cdots \times 10^{n-k}}{0.d_1 d_2 \cdots d_k \times 10^n} \right| \leqslant \frac{1}{0.1} \times 10^{-k} = 10^{1-k}.$$

So 10^{1-k} is a error bound for the relative error of $fl(y)$.

Similarly, $0.5 \times 10^{1-k}$ is a error bound for the relative error of $fl(y)$ when k-decimal rounding is used.

1.1.3 Finite Digit Arithmetic

Assume that $x, y \in \mathbb{R}$, define symbols $\oplus, \ominus, \otimes, \oslash$ for the finite digit arithmetic as follows:

$$x \oplus y = fl(fl(x) + fl(y)), \qquad x \ominus y = fl(fl(x) - fl(y)),$$
$$x \otimes y = fl(fl(x) \cdot fl(y)), \qquad x \oslash y = fl\left(\frac{fl(x)}{fl(y)}\right).$$

Some calculation would produce a large error. For example, the cancellation of significant digits due to the subtraction of nearly equal numbers. Suppose $x > y$ have the k-digit representations:

$$fl(x) = 0.d_1 d_2 \cdots d_p \alpha_{p+1} \cdots \alpha_k \times 10^n,$$
$$fl(y) = 0.d_1 d_2 \cdots d_p \beta_{p+1} \cdots \beta_k \times 10^n,$$

where $\alpha_{p+1} > \beta_{p+1}$. Then we have

$$\begin{aligned} x \ominus y &= fl(fl(x) - fl(y)) \\ &= fl\left(0.00 \cdots 0 \sigma_{p+1} \cdots \sigma_k \times 10^n\right) \\ &= 0.\sigma_{p+1} \cdots \sigma_k \times 10^{n-p}, \end{aligned}$$

which has at most $k - p$ significant digits. In any calculation device, the last few digits of $x - y$ would be assigned either zero or some random digits, which may lead to a large relative error in afterwards calculations (for example, dividing by a small number).

Sometimes, the loss of accuracy can be avoided by reformulating the calculation. For example, to compute the root of a quadratic equation $ax^2 + bx + c = 0$ with the root formula:

Example: **Reformulation of Calculation**

Consider $x^2 + 62.1x + 1 = 0$. The roots are given by

$$x_1 \approx -0.01610723, \qquad x_2 \approx -62.08389277.$$

Note that directly calculating x_1 would cause a large error due to the cancellation of significant digits. Instead, we can calculate x_1 by a reformulation of the root formula:

$$x_1 = \frac{-b - \sqrt{b^2 - 4ac}}{2a} = \frac{-2c}{b + \sqrt{b^2 - 4ac}}$$

which would result in a much smaller relative error, from 2.4×10^{-1} to 6.2×10^{-4} for a 4-digit decimal machine number.

We can do some approximations on the relative errors as follows: Let x^*, y^* be the approximations of x, y respectively, with relative errors e_x, e_y. Then we have

- Addition and Subtraction: $\dfrac{|(x \pm y) - (x^* \pm y^*)|}{|x \pm y|} \leqslant \dfrac{|x|e_x + |y|e_y}{|x \pm y|}$.

1.1.4 Nested Arithmetic

This is a way to calculate a polynomial:

Consider the polynomial

$$p(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n. \tag{1.4}$$

We can calculate $p(x)$ by the following nested form:

$$p(x) = a_0 + x(a_1 + x(a_2 + \cdots + x(a_{n-1} + xa_n)\cdots)). \tag{1.5}$$

This reduces the number of multiplications from $n(n+1)/2$ to n, and thus significantly reduces the round-off error.

Remark:
Polynomials should always be expressed in nested form before performing an evaluation because this form minimizes the number of arithmetic calculations. This shows that one way to reduce round-off error is to reduce the number of computations.

1.2 Algorithms and Convergence

Definition 1.2.1: Algorithm

An **algorithm** is a finite sequence of instructions that, if followed, accomplishes a particular task. The task may be to solve a mathematical problem, or to perform a computation.

We use a pseudocode to describe the algorithms.

1.2.1 Characterization of Algorithms

A **stable** algorithm is one that produces correspondingly small changes in the output for small changes in the input. Some algorithm are **conditionally stable**, which means that the algorithm is stable for some inputs but not for others.

To quantify this idea, suppose an error with magnitude $E_0 > 0$ is introduced at some stage of the algorithm, and E_n is the magnitude of the error after n steps, then:

- If $E_n \approx CnE_0$ for some constant C, then the growth of the error is **linear**.
- If $E_n \approx C^nE_0$ for some constant $C > 1$, then the growth of the error is **exponential**.

Linear growth of error is usually unavoidable, and when C and E_0 are small, the results are generally acceptable. Exponential growth of error should be avoided because the term C^n becomes large for even relatively small values of n. This leads to unacceptable inaccuracies, regardless of the size of E_0. As a consequence, an algorithm that exhibits linear growth of error is stable, whereas an algorithm exhibiting exponential error growth is unstable.

Remark:
Usually when we analyze the stability of an algorithm, we only assume the loss of accuracy

occurs at the beginning of the algorithm (when input data is read into the computer), and neglect the round-off error produced by the arithmetic operations in the algorithm. This is a simplified model of the error, and the actual error is likely to be larger.

1.2.2 Rate of Convergence

Definition 1.2.2: Rate of Convergence

Suppose $\{\beta_n\}$ is a sequence converging to zero and $\{\alpha_n\}$ converges to α. If there exists a constant $K > 0$ such that

$$|\alpha_n - \alpha| \leqslant K|\beta_n|$$

for large n, then we say $\{\alpha_n\}$ converges to α with **rate of convergence** $O(\beta_n)$, and write $\alpha_n = \alpha + O(\beta_n)$.
Similarly for functions, if $\lim_{h\to 0} G(h) = 0$ and $\lim_{h\to 0} F(h) = L$, and there exists a constant $K > 0$ such that

$$|F(h) - L| \leqslant K|G(h)|$$

for small h, then we say $F(h)$ converges to L with **rate of convergence** $O(G(h))$, and write $F(h) = L + O(G(h))$.

Remark:
Generally we have the following conclusions for a good approximation algorithm:

- Avoid subtraction of nearly equal numbers. (Increase relative error.)
- Avoid division by small numbers. (Increase Absolute error.)
- Avoid adding a small number to a large number.
- Reduce the number of arithmetic calculations. (Increase round-off error.)
- Alogrithms should be stable.

Chapter 2

Numerical Solution of Linear Systems

Finding the solution of a linear algebraic equation system of large order and calculating the inverse of a matrix of large order can be difficult numerical tasks. The difficulties are practical and stem from

- the labor required in a lengthy sequence of calculations.
- the possible loss of accuracy in such lengthy calculations performed with a fixed number of decimal places.

Thus to determine the feasibility of solving a particular problem with given equipment, several questions should be answered:

- How many arithmetic operations are required to apply a proposed method?
- What will be the accuracy of a solution to be found by the proposed method (a priori estimate)?
- How can the accuracy of the computed answer be checked (a posteriori estimate)?

In the chapter we mainly discuss two problems:

- the solution of a linear system of equations $Ax = b$.
- the calculation of the inverse of a matrix A^{-1}.

These two problems are closely related: If A^{-1} is known, then the solution of $Ax = b$ can be found by a single matrix-vector multiplication $x = A^{-1}b$. Conversely, if the solution of $Ax = b$ can be found for any b, then the inverse of A can be found by solving n systems of equations $Ax = e_i$, where e_i is the i-th column of the identity matrix.

2.1 Gaussian Elimination

Consider the system of linear equations

$$Ax = f \tag{2.1}$$

where $A = (a_{ij})$ is an $n \times n$ matrix, $x = (x_1, x_2, \ldots, x_n)^T$ is the unknown vector, and $f = (f_1, f_2, \ldots, f_n)^T$ is the known vector.

Giving the system of equations, we perform the row operations to transform the system into an upper triangular form: Let the equation of k-th step be $A^{(k)}x = f^{(k)}$. We perform the following row operations:

$$a_{ij}^{(k)} = \begin{cases} a_{ij}^{(k-1)}, & i \leqslant k-1 \\ 0, & i \geqslant k, j \leqslant k-1 \\ a_{ij}^{(k-1)} - \frac{a_{i,k-1}^{(k-1)}}{a_{k-1,k-1}^{(k-1)}} a_{k-1,j}^{(k-1)}, & i \geqslant k, j \geqslant k \end{cases} \tag{2.2}$$

and for the right-hand side vector,

$$f_i^{(k)} = \begin{cases} f_i^{(k-1)}, & i \leqslant k-1 \\ f_i^{(k-1)} - \frac{a_{i,k-1}^{(k-1)}}{a_{k-1,k-1}^{(k-1)}} f_{k-1}^{(k-1)}, & i \geqslant k \end{cases} \tag{2.3}$$

These formulae represent the result of multiplying the $(k-1)$-th equation by $a_{i,k-1}^{(k-1)}/a_{k-1,k-1}^{(k-1)}$ and subtracting it from the i-th equation, for $i \geqslant k$. After $n-1$ steps, we have an upper triangular system of equations. It has been assumed above that the elements $a_{k-1,k-1}^{(k-1)}$ are nonzero for $k = 1, 2, \ldots, n-1$. This holds if the matrix A is nonsingular.

If this is not the case, we can perform row interchanges to make the pivot element nonzero.

Theorem 2.1.1: Gauss Elimination

Let A be be such that the Gaussian elimination procedure in natural order can be performed resulting in an upper triangular system of equations. Then A is nonsigular and in fact

$$\det A = a_{11}^{(1)} a_{22}^{(2)} \cdots a_{nn}^{(n)}. \tag{2.4}$$

The final matrix $A^{(n)} = U$ is an upper triangular matrix, and A has the LU factorization $A = LU$, where $L = m_{ik}$ is a unit lower triangular matrix with entries

$$m_{ik} = \begin{cases} 0, & i < k \\ 1, & i = k \\ a_{ik}^{(k)}/a_{kk}^{(k)}, & i > k \end{cases}$$

The final vector $f^{(n)} = g$ is $g = L^{-1}f$.

The system is equivalent to $Ux = g$, which can be solved by back substitution:

$$x_n = g_n/u_{nn}, \quad x_i = (g_i - \sum_{j=i+1}^{n} u_{ij}x_j)/u_{ii}, \quad i = n-1, n-2, \ldots, 1.$$

Theorem 2.1.2: Gauss Elimination with Pivoting

Let the matrix A has rank r. Then we can find a sequence of distinct row and column indices $(i_1, j_1), (i_2, j_2), \ldots, (i_r, j_r)$ such that the corresponding pivot elements in $A^{(1)}, A^{(2)}, \ldots, A^{(r)}$ are nonzero and that $a_{ij}^{(r)} = 0$ for $i \notin \{i_1, i_2, \ldots, i_r\}$. Let us define the permutation matrices, whose columns are unit vectors,

$$P = (e^{(i_1)}, e^{(i_2)}, \ldots, e^{(i_r)}, e^{(i_{r+1})}, \ldots, e^{(i_n)}), \quad Q = (e^{(j_1)}, e^{(j_2)}, \ldots, e^{(j_r)}, e^{(j_{r+1})}, \ldots, e^{(j_n)}).$$

where $i_1, i_2, \ldots, i_n$ and $j_1, j_2, \ldots, j_n$ are permutations of $1, 2, \ldots, n$. Then the system

$$By = g, \qquad B = P^T AQ, y = Q^T x, g = P^T f$$

is equivalent to the original system $Ax = f$ and can be reduced to an upper triangular system of equations by Gaussian elimination without pivoting.

In actual computations on digital computers it is a simple matter to record the order of the pivot indices (i_ν, j_ν) and to do the arithmetic accordingly.

Remark:
One way to pick the pivots is to require that (i_k, j_k) be the indices of a maximal coefficient in the system of $n - k + 1$ equations that remain at the k-th step. This method of selecting maximal pivots is recommended as being likely to introduce the least loss of accuracy in the arithmetical operations that are based on working with a finite number of digits.

Another commonly used pivotal selection method eliminates the variables $x_1, \ldots, x_{n-1}$ successively by selecting $(i_k, j_k) = (i_k, k)$, where i_k is the index of the largest element in the k-th column of the system of $n - k + 1$ equations that remain at the k-th step. This method of maximal column pivots is particularly convenient for use on an electronic computer if the large matrix of coefficients is stored by columns since the search for a maximal column element is then quicker than the maximal matrix element search.

2.1.1 Operational Counts

If the $n \times n$ matrix A is nonsingular, Gaussian elimination might be employed to solve the n special linear systems and thus to obtain A^{-1}. *However, we shall show here that for any value of m this procedure is less efficient than an appropriate application of Gaussian elimination to the m systems in question.*

The current convention is to count only multiplications and divisions. This custom arose because the first modern digital computers performed additions and subtractions much faster than they did multiplications and divisions which were done in comparable lengths of time.

Let us consider first the m systems, with arbitrary $f(j)$, where $j = 1, 2, \ldots, m$. We assume, for the operational count, that the elimination proceeds in the natural order. The most efficient application then performs the operations in equations (2.2) once for all m systems, and once for each of the m systems for the operations in equations (2.3). Thus, we find that it requires

$$\begin{aligned} (n-k+1)^2 + (n-k+1) \quad & \text{ops. for (2.2)} \\ (n-k+1) \quad & \text{ops. for (2.3)} \end{aligned}$$

to eliminate x_{k-1}. Thus, the total number of operations for the elimination phase is given by summing over k:

$$\begin{aligned} \frac{1}{3}n(n^2-1) \quad & \text{ops. for triangularization} \\ \frac{1}{2}n(n-1) \quad & \text{ops. for solving one inhomogeneous vector } f(j) \end{aligned}$$

To solve the resulting triangular system, we need $(n-i)$ multiplications and one division for x_i. So we have

$$\frac{1}{2}n(n+1) \qquad \text{ops. for back substitution}$$

In total we have

Lemma 2.1.1: Operational Count for Gaussian Elimination

The total number of multiplications and divisions required to solve m systems of linear equations by Gaussian elimination is

$$\frac{1}{3}n^3 + mn^2 - \frac{1}{3}n \tag{2.5}$$

To compute A^{-1}, just set $m = n$ and we get

$$\frac{4}{3}n^3 - \frac{1}{3}n \qquad \text{ops. for computing } A^{-1} \text{ by Gaussian elimination}$$

However, here the n inhomogeneous vectors are $e^{(j)}$, which is special. So we can reduce the number of operations with some tactics. That is, for any fixed $1 \leqslant j \leqslant n$, the calculations to be counted in (2.3) can start at $i = j$, which is $k = j + 2$. Thus, we have

$$\sum_{k=j+2}^{n} (n-k+1) = \frac{1}{2}(j^2 - (2n-1)j + n^2 - n)$$

to modify the inhomogeneous vector $e^{(j)}$ for $j = 1, 2, \ldots, n-2$, we have

$$\frac{1}{6}(n^3 - 3n^2 + 2n) \qquad \text{ops. for modifying the inhomogeneous vectors}$$

The other operations are the same as before, so we have

Lemma 2.1.2: Operational Count for Inversion

The total number of multiplications and divisions required to compute A^{-1} by Gaussian elimination is

$$n^3 \tag{2.6}$$

Thus, to solve m systems by inversion, we need

$$n^3 + mn^2 \qquad \text{ops. for solving } m \text{ systems by inversion}$$

2.1.2 A Priori Error Estimates and Condition Number

In the course of actually carrying out the arithmetic required to solve $Ax = f$ by any procedure, round-off errors will in general be introduced.

We recall that a computation is said to be well-posed if "small" changes in the data cause "small" changes in the solution. Now the data is the matrix A and the vector f, and the solution is the vector x. Suppose we have a perturbation:

$$(A + \delta A)(x + \delta x) = f + \delta f$$

Theorem 2.1.3: Condition Number

Let A be a nonsingular matrix. And

$$\|\delta A\| < \frac{1}{\|A^{-1}\|}$$

Then we have

$$\frac{\|\delta x\|}{\|x\|} \leqslant \frac{\mu}{1-\mu\delta A/\|A\|}\left(\frac{\|\delta A\|}{\|A\|}+\frac{\|\delta f\|}{\|f\|}\right) \tag{2.7}$$

where $\mu = \|A\|\|A^{-1}\|$ is called the condition number of A.

Proof. Since $\|A^{-1}\delta A\| \leqslant \|A^{-1}\|\|\delta A\| < 1$, we have $I + A^{-1}\delta A$ is nonsingular. Thus,

$$\|(I+A^{-1}\delta A)^{-1}\| \leqslant \frac{1}{1-\|A^{-1}\delta A\|} \leqslant \frac{1}{1-\mu\delta A/\|A\|}$$

from Taylor expansion. We also have

$$\delta x = (I+A^{-1}\delta A)^{-1}A^{-1}(\delta f - \delta A x)$$

Thus,

$$\frac{\|\delta x\|}{\|x\|} \leqslant \frac{\|A^{-1}\|}{1-\mu\delta A/\|A\|}\left(\frac{\|\delta f\|}{\|x\|}+\|\delta A\|\right)$$

Note that

$$\|f\| = \|Ax\| \leqslant \|A\|\|x\|$$

we have

$$\frac{\|\delta x\|}{\|x\|} \leqslant \frac{\mu}{1-\mu\delta A/\|A\|}\left(\frac{\|\delta A\|}{\|A\|}+\frac{\|\delta f\|}{\|f\|}\right)$$

giving the desired result. □

It shows that small relative changes in A and f would cause small relative changes in x if the factor

$$\frac{\mu}{1-\mu\delta A/\|A\|}$$

is not too large. Thus, it is clear that when the condition number μ is not too large, the system is well-conditioned. However, we shall note that there is an lower bound for μ:

$$\|I\| = \|AA^{-1}\| \leqslant \|A\|\|A^{-1}\| = \mu$$

We can apply the theorem to estimate the effects of round-off errors committed in solving linear systems by Gaussian elimination and other direct methods. But it is possible to view the computed solution as the exact solution, say $x+\delta x$, of a perturbed system, and the basic problem now is to determine the magnitude of the perturbations, δA and δf. This type of approach is called a *backward error analysis.* It is rather clear that there are many perturbations δA and δf that give the same result. Our analysis for the Gaussian elimination method will define δA and δf in a way that

$$\delta f = 0.$$

So that

$$\frac{\|\delta x\|}{\|x\|} \leqslant \frac{\mu\|\delta A\|/\|A\|}{1-\mu\|\delta A\|/\|A\|}$$

In the case of Gaussian elimination we have seen that exact calculations yield the factorization

$$LU = A$$

Now, lets consider the effect of round-off errors in the Gaussian elimination procedure from the perspective of LU factorization. The Gaussian elimination procedure can be viewed as computing $g = L^{-1}f$ and the back substitution as computing $x = U^{-1}g$.

The Gaussian elimination procedure In the phase, we calculate the entries of L and U by the formulae in (2.2). With exact calculations, we have $LU = A$. However, with finite precision arithmetic in these evaluations, we do not obtain exactly L and U, but say some triangular matrices $\mathcal{L}$ and $\mathcal{U}$. We define the perturbation E due to inexact calculations by

$$\mathcal{L}\mathcal{U} = A + E$$

The back substitution In the phase, we calculate the entries of x by the formulae in (2.3). With exact calculations, we have $Lg = f$ and $Ux = g$. However, with finite precision arithmetic, we have got $g + \delta g$ and $x + \delta x$ such that

$$\begin{aligned}(\mathcal{L} + \delta\mathcal{L})(g + \delta g) &= f,\\ (\mathcal{U} + \delta\mathcal{U})(x + \delta x) &= g + \delta g\end{aligned}$$

Here we use the $\mathcal{L}$ and $\mathcal{U}$ obtained in the Gaussian elimination procedure. We have

$$A + \delta A = (\mathcal{L} + \delta\mathcal{L})(\mathcal{U} + \delta\mathcal{U}), \qquad \delta A = E + \mathcal{L}(\delta\mathcal{U}) + (\delta\mathcal{L})\mathcal{U} + (\delta\mathcal{L})(\delta\mathcal{U})$$

Remark:
We can say that L and U took round-off errors both in the Gaussian elimination procedure and the back substitution, the first phase gives inexact $\mathcal{L}$ and $\mathcal{U}$, and the second phase gives additional perturbations $\delta\mathcal{L}$ and $\delta\mathcal{U}$ here.
This tactic is useful in analyzing algorithm errors: For inexact solution $Ax = f$, we say that the result we got $x + \delta x$ is the exact solution of a perturbed system $(A + \delta A)(x + \delta x) = f$, where we ASSUME (entirely mathematical model) that f is not perturbed and accurate, to eliminate a possible degree of freedom in the choice of δA and δf. Then we can analyze the error by estimating $\|\delta A\|$ and applying the condition number theorem.

Now, we must estimate $\|E\|, \|\delta\mathcal{L}\|$ and $\|\delta\mathcal{U}\|$ for the actual computations.

We shall assume that floating-point arithmetic operations are performed with a t-digit decimal mantissa and that the system has been ordered so that the natural order of pivots is used.

For a convenient notation, in place of the accurate matrices $A^{(k)}$ defined by (2.2), we shall use the matrices $B^{(k)} = (b_{ij}^{(k)})$ with the final such matrix $B^{(n)} = \mathcal{U} = (u_{ij})$. Similarly, the lower triangular matrix $\mathcal{L} = (s_{ij})$ is given by the computed multipliers. For simplicity, we assume that the given matrix elements a_{ij} are exactly representable in the floating-point system.

The floating-point calculations yield

- For $k = 1$,
$$b_{ij}^{(1)} = a_{ij} \qquad i, j = 1, 2, \ldots, n$$
- For $k = 1, 2, \ldots, n-1$,
$$b_{ij}^{(k+1)} = \begin{cases} b_{ij}^{(k)}, & i \leqslant k \\ 0, & i \geqslant k+1, j \leqslant k \\ (b_{ij}^{(k)} - s_{ik} b_{kj}^{(k)}(1 + \theta_{ij}^{(k)} 10^{-t}))(1 + \phi_{ij}^{(k)} 10^{-t}), & i \geqslant k+1, j \geqslant k+1 \end{cases}$$

$$s_{ij} = \begin{cases} 0, & i \leqslant j \\ 1, & i = j \\ \frac{b_{ij}^{(j)}}{b_{jj}^{(j)}}(1 + \psi_{ij} 10^{-t}), & i > j \end{cases}$$

here
$$|\theta_{ij}^{(k)}| \leqslant 5, \quad |\phi_{ij}^{(k)}| \leqslant 5, \quad |\psi_{ij}| \leqslant 5$$
and they account for the rounding procedures in floating-point arithmetic. The above calculations can be carried out iff the $b_{jj}^{(j)}$ are nonzero for $j \leqslant n-1$. This can be assured by the following lemma.

Lemma 2.1.3: Numerical Gaussian Elimination

If A is non-singular and t is sufficiently large, then the Gaussian elimination method, with maximal column pivots and floating-point arithmetic (with t-digit mantissas), yields multipliers s_{ij} with $|s_{ij}| \leqslant 1$ and pivots $b_{jj} \neq 0$.

It turns out that we require a bound on the growth of the pivot elements for our error estimates. That is, we seek a quantity $G(n)$, independent of the a_{ij}, such that
$$|b_{jj}^{(j)}| \leqslant G(n)a, \qquad a = \max_{i,j} |a_{ij}| \tag{2.8}$$
It is not difficult to see that
$$G(n) \leqslant (1 + (1 + 5 \times 10^{-t})^2)^{n-1} = 2^{n-1} + \mathcal{O}((n-1)10^{1-t}) \tag{2.9}$$
This establishes the existence of a bound, but it is a tremendous overestimate for large n. In fact for exact elimination, using maximal pivots it can be shown that
$$|a_{jj}^{(j)}| < g(j)a, \qquad g(j) \leqslant 3j^{1/2 + 1/4 \ln j} \tag{2.10}$$
The quantity $g(n)$ would be a reasonable estimate for $G(n)$ if the maximal pivots in the sequence $\{B^{(k)}\}$ were located in the same positions as the maximal pivots in $\{A^{(k)}\}$. We know that if A is non-singular and t is sufficiently large, then the indices of the maximal pivotal elements used to find $\{B^{(k)}\}$ are also indices of maximal pivots in an exact Gaussian elimination procedure for A. (because of continuity)

The best lower bound for $G(n)$ is not known yet.

We now turn to estimates of the terms in δA. Our first result is a bound on the elements of E:

Proposition: **Bounds for E**

Under the hypotheses of lemma 2.1.3, the elements of E satisfy

$$|e_{ij}| \leqslant \begin{cases} (i-1)2aG(n)10^{1-t}, & i \leqslant j \\ j2aG(n)10^{1-t}, & i > j \end{cases} \tag{2.11}$$

For any G satisfying (2.8).

Proof. We can write

$$b_{ij}^{(k+1)} = b_{ij}^{(k)} - s_{ik}b_{kj}^{(k)} + \epsilon_{ij}^{(k+1)}, \qquad i \geqslant k+1, j \geqslant k+1 \tag{2.12}$$

where

$$\epsilon_{ij}^{(k+1)} = b_{ij}^{(k)}\phi_{ij}^{(k)}10^{-t} - s_{ik}b_{kj}^{(k)}(\theta_{ij}^{(k)} + \phi_{ij}^{(k)})10^{-t} - \cdots$$

For maximal pivots, we have $|s_{ik}| \leqslant 1$ and $|b_{ij}^{(k)}| \leqslant G(n)a$, so we have

$$|\epsilon_{ij}^{(k+1)}| \leqslant 2aG(n)10^{1-t}$$

For the second equation

$$s_{ij} = \frac{b_{ij}^{(j)}}{b_{jj}^{(j)}}(1 + \psi_{ij}10^{-t}), \qquad i > j$$

rewrite it as

$$0 = b_{ij}^{(k)} - s_{ij}b_{jj}^{(j)} + \epsilon_{ij}^{(j+1)}, \qquad i > j$$

where

$$\epsilon_{ij}^{(j+1)} = s_{ij}b_{jj}^{(j)}\psi_{ij}10^{-t}$$

where again we find that

$$|\epsilon_{ij}^{(j+1)}| \leqslant 2aG(n)10^{1-t}$$

Now, we can write $\mathcal{L} = (s_{ij})$ and $\mathcal{U} = (b_{ij}^{(i)})$. We have

$$(\mathcal{LU})_{ij} = \sum_{k=1}^{n} s_{ik}b_{kj}^{(k)} = \sum_{k=1}^{\min(i,j)} s_{ik}b_{kj}^{(k)}$$

For $i > j$, as we have

$$0 = b_{ij}^{(1)} - \sum_{k=1}^{j} s_{ik}b_{kj}^{(k)} + \sum_{k=1}^{j} \epsilon_{ij}^{(k+1)}$$

from the first j steps of the Gaussian elimination procedure (2.12), we have

$$e_{ij} = \sum_{k=1}^{j} \epsilon_{ij}^{(k+1)}, \qquad i > j$$

For $i \leqslant j$, we have

$$e_{ij} = \sum_{k=1}^{i-1} \epsilon_{ij}^{(k+1)}, \qquad i \leqslant j$$

giving the desired bounds. □

As a simple corollary of this theorem, we note that since $|e_{ij}| \leqslant 2naG(n)10^{1-t}$, we have

$$\|E\|_\infty \leqslant 2an(n-1)G(n)10^{1-t}$$

The elements in $\delta\mathcal{L}$ and $\delta\mathcal{U}$ can be estimated from a single analysis of the error in solving any triangular system (with the same arithmetic and rounding). Thus we consider, say, $Tu = h$, where $T = (t_{ij})$ is lower triangular and nonsingular. Then we have

$$u_1 = t_{11}^{-1}h_1, \quad u_i = t_{ii}^{-1}\left(h_i - \sum_{j=1}^{i-1} t_{ij}u_j\right), \quad i = 2, 3, \ldots, n \tag{2.13}$$

Proposition: **Bounds of Back Substitution Errors**

Let the solution of (2.13) be computed with floating-point arithmetic with t-digit mantissas, then the computed solution v satisfies $(T + \delta T)v = h$, where δT is bounded by

$$|\delta t_{ij}| \leqslant \max\{2, |i - j + 1|\}|t_{ij}|10^{1-t} \leqslant n|t_{ij}|10^{1-t}$$

Here t is required to be sufficiently large so that $n10^{1-t} \leqslant 1$.

We are now able to obtain estimates of the elements in $\delta\mathcal{L}$ and $\delta\mathcal{U}$, and thus of δA.

Theorem 2.1.4: Wilkinson Error Estimate

Let the n-th order matrix A be non-singular and employ Gaussian elimination with maximal pivots and t-digit floating point arithmetic to solve $Ax = f$. Let t be sufficiently large so that $n10^{1-t} \leqslant 1$. Then the computed solution $x + \delta x$ satisfies

$$(A + \delta A)(x + \delta x) = f$$

where

$$|\delta a_{ij}| \leqslant (2n + 3n^2)aG(n)10^{1-t} \tag{2.14}$$

and $G(n)$ is any bound satisfying (2.8).

Proof. SORRY. □

From this theorem, it follows that the computed solution is the exact solution of a system only slightly perturbed from the original if enough figures are used. Appropriate values for t depend upon n and the bound $G(n)$. If $G(n) \sim 2^{n-1}$, only relatively small order systems could be treated effectively. On the other hand, if $G(n) \sim g(n) \leqslant 3n^{1/2+1/4\ln n}$, then quite large order systems can be treated with the number of digits used on modern digital computers, say $t \geqslant 8$. It is generally believed, however, that even this latter estimate for $G(n)$ is a gross overestimate when using maximal pivots.

It should be observed that essentially all of the previous analysis is valid if only partial pivoting, say maximal column pivoting, is employed since then $|s_{ij}| \leqslant 1$ is maintained. However, the growth factor $G(n)$ for this procedure cannot be estimated well in general. In fact, it is possible that the upper bound (2.9) which still applies may be attained. In spite of this, partial pivoting is found to be effective in practice but the absence of any type of maximal pivoting strategy frequently leads to catastrophic growth of rounding errors.

We can easily see, from 2.1.4, that

$$\|\delta A\|_\infty \leqslant (2n + 3n^2)aG(n)10^{1-t}$$

and this can be employed to obtain maximum norm bounds on the relative error. It is clear that this relative error in x may not be small even though the relative perturbation $\|\delta A\|/\|A\|$ is small if the condition number μ is large. For a given $G(n)$ and $\mu(A)$, we can determine the required number of digits t to ensure that the computed solution is accurate to a specified number of digits.

Finally, we recall that a computed inverse of A can be obtained by solving n systems. Denote the calculated inverse by $A^{-1} + F$, then as above we can show that each column vector of it satisfies an equation of the form

$$(A + \delta A^{(j)})(A^{-1} + F)_j = e^{(j)}$$

If the condition of theorem 2.1.4 is satisfied, then similarly, we have

$$\frac{\|F_j\|}{\|A_j^{-1}\|} \leqslant \frac{\mu(A)\|\delta A^{(j)}\|/\|A\|}{1 - \mu(A)\|\delta A^{(j)}\|/\|A\|} \tag{2.15}$$

2.1.3 A Posteriori Error Estimates

Although we do not advocate inverting a matrix to solve linear systems, it is of interest to consider error estimates related to computed inverses.

Let A be the matrix to be inverted and let C be the computed or alleged inverse, then let $F = C - A^{-1}$ be the error. We also define the **residue matrix** R by

$$R = AC - I \tag{2.16}$$

Then we have

Theorem 2.1.5: Residue Matrix

If $\|R\| < 1$, then

- A and C are nonsingular;
- $\|A^{-1}\| \leqslant \|C\|/(1 - \|R\|)$, and $\|C^{-1}\| \leqslant \|A\|/(1 - \|R\|)$;
- $\|F\| \leqslant \|C\|\|R\|/(1 - \|R\|)$, and $\|A - C^{-1}\| \leqslant \|A\|\|R\|/(1 - \|R\|)$.

Proof. SORRY. □

We can also find the perturbation δA such that C is the exact inverse of $A + \delta A$. We have

$$(A + \delta A)C = I, \qquad \delta A = -RC^{-1}$$

Taking norms we have

$$\|\delta A\| \leqslant \|R\|\|C^{-1}\| \leqslant \frac{\|A\|\|R\|}{1 - \|R\|}$$

Finally, we observe that the computed inverse can also be used to estimate the error in solving a linear system. We state this result as

Theorem 2.1.6: Error Estimate by Computed Inverse

Let an approximated solution y of $Ax = f$ have the **residue vector** $r = Ay - f$. If an approximated inverse C of A has the residue matrix $\|R\| = \|AC - I\| < 1$, then

$$\|y - x\| \leqslant \frac{\|C\|\|r\|}{1 - \|R\|}$$

Remark:
It should be noted that the result in this theorem is independent of the manner in which the approximate solution y is obtained. Thus, it was not assumed that $y = Cf$.

2.2 Variants of Gaussian Elimination

There are many methods for solving linear systems that are slight variations of the Gaussian elimination method. None of these methods has succeeded in reducing the number of operations required, but some have eliminated much of the intermediate storage or recording requirements.

Gauss-Jordan elimination The modification due to Jordan circumvents the final back substitution. This is accomplished by eliminate not only the entries below the pivot elements but also those above them in the same column. Thus, the resulting system is diagonal and the solution can be read off directly:

- $a_{ij}^{(1)} = a_{ij}$ for $i, j = 1, 2, \ldots, n$;
- $a_{ij}^{(k)} = a_{ij}^{(k-1)} - \dfrac{a_{i,k-1}^{(k-1)}}{a_{k-1,k-1}^{(k-1)}} a_{k-1,j}^{(k-1)}$ for $i \neq k-1, j \geqslant k-1$;
- $a_{k-1,j}^{(k)} = a_{k-1,j}^{(k-1)}$ for $j \geqslant k-1$; and $a_{ij}^{(k)} = a_{ij}^{(k-1)}$ for $j < k-1$.

Also f takes the same modification as in Gaussian elimination. The result is simply

$$x_i = \frac{f_i^{(n)}}{a_{ii}^{(n)}} = \frac{f_i^{(n)}}{a_{ii}^{(i)}}, \qquad i = 1, 2, \ldots, n$$

It is clear that pivoting on the maximal element in the remaining square submatrix may be retained in this procedure. Hence, multipliers for $i < k-1$ may exceeds unity. Furthermore, the number of operations is somewhat greater than in the ordinary Gaussian elimination with back substitution; it is now

$$\frac{1}{2}n^3 + n^2 - \frac{1}{2}n \qquad \text{ops. for Jordan elimination} \tag{2.17}$$

Thus, there does not seem to be any great advantage in using the Jordan elimination in actual calculations with automatic computing equipment.

Crout Reduction This method is applicable if *the rows and columns are so arranged that no column interchanges are required in the Gaussian elimination procedure.*

Thus, in general, the pivots will not be the maximal elements. Hence, errors may grow very rapidly in the Crout method and it is not recommended unless the system is of relatively small order or if it can be determined that the error growth will not be catastrophic. (In practice one may apply the method and test the accuracy of the solution a posteriori.)

On the other hand, the Crout method is specifically designed to *reduce the number of intermediate quantities which must be retained.* Thus, for hand computations and digital computers with small storage capacities it may be of great value.

Consider the factorization

$$LU = A$$

where $L = (m_{ij})$ and $U = (u_{ij})$ are lower triangular matrix with unit diagonal elements and upper triangular matrix. Writing the formula for a_{ij}, we have

$$a_{ij} = \sum_{k=1}^{\min(i,j)} m_{ik}u_{kj}$$

Thus, we have

$$u_{ij} = a_{ij} - \sum_{k=1}^{i-1} m_{ik}u_{kj}, \qquad i \leqslant j \tag{2.18}$$

$$m_{ij} = \frac{1}{u_{jj}}\left(a_{ij} - \sum_{k=1}^{j-1} m_{ik}u_{kj}\right), \qquad i > j \tag{2.19}$$

The actual elimination procedure is as follows:

1. Compute the first row of U by simply copying the first row of A.
2. Compute the first column of L by dividing the first column of A: $m_{i1} = a_{i1}/u_{11}$ for $i = 2, 3, \ldots, n$.
3. Compute the second row of U by using the first column of L: $u_{2j} = a_{2j} - m_{21}u_{1j}$ for $j = 2, 3, \ldots, n$.
4. Compute the second column of L by using the second row of U: $m_{i2} = (a_{i2} - m_{i1}u_{12})/u_{22}$ for $i = 3, 4, \ldots, n$.
5. The procedure continues in this way, alternately computing rows of U and columns of L until the entire factorization is obtained.

If we append the inhomogeneous vector f to the right of A and apply the above procedure to the resulting augmented matrix, then we can obtain g as the rightmost column of the resulting upper triangular matrix U.

The operational count, for producing L, U and g is easily found to be

$$\frac{1}{3}n^3 + \frac{1}{2}n^2 - \frac{5}{6}n \qquad \text{ops. for Crout reduction}$$

It is not surprising that this is the same as the number of operations required by the conventional Gaussian elimination scheme.

Remark:
This "compact" elimination procedure is based on the fact that only those elements $a_{ij}^{(k)}$ in the Gaussian elimination, for which $j \geqslant i$ and $i \leqslant k$, are required for the final back substitution.

We could show now, if the inner products in **??** and **??** are evaluated in *double precision* before the sum is rounded, that the effect of rounding errors is appreciably diminished. In fact, the estimate becomes

$$\|\delta A\|_\infty = \mathcal{O}(n^2 G(n) a 10^{-t}) \tag{2.20}$$

While Gaussian Elimination and Crout Reduction appear to be different algorithms, they are mathematically equivalent ways of achieving the same goal: the LU **Factorization** of a matrix A.

If no row or column interchanges (pivoting) are performed, both methods produce the exact same matrices L and U.

- **Gaussian Elimination** is a *procedural* approach that transforms A into an upper triangular matrix U through row operations. The multipliers used during this process form the lower triangular matrix L.
- **Crout Reduction** is a *compact* approach that directly solves the matrix equation $A = LU$ for the unknown elements of L and U.

The core connection lies in how the elements are updated. In Gaussian Elimination, an element a_{ij} is updated step-by-step. After k steps, the updated element is:

$$a_{ij}^{(k)} = a_{ij} - \sum_{p=1}^{k} m_{ip} u_{pj}$$

In the **Crout Method**, we do not store these intermediate $a_{ij}^{(k)}$ values. Instead, we compute the final sum all at once. Looking at the Crout formula for u_{ij}:

$$u_{ij} = a_{ij} - \sum_{k=1}^{i-1} m_{ik} u_{kj}$$

This is exactly the value that Gaussian Elimination would have produced in the i-th row after all previous rows had been eliminated.

The difference is purely the **order of operations**:

Gaussian Elimination (Incremental): Uses the k-th pivot to update *all* remaining elements in the sub-matrix below and to the right. It is "breadth-first."

Crout Reduction (Compact): Calculates the final value of one row of U and one column of L at a time. It is "depth-first."

Even though the operational count ($\frac{1}{3}n^3$) is identical, the Crout method is often preferred in specific contexts because:

1. **Inner Products:** It expresses the calculation as a series of inner products (dot products).

2. **Precision:** As noted in the text, evaluating these inner products in *double precision* before rounding minimizes the accumulation of rounding errors compared to the step-by-step subtractions in Gaussian Elimination.

3. **Storage:** It reduces the need to store intermediate "modified" matrices $A^{(k)}$.

Remark:
In summary, Crout Reduction is simply a *re-ordering* of the arithmetic of Gaussian Elimination designed to optimize memory usage and numerical stability via high-precision accumulation.

2.3 Direct Factorization Methods

Now, we perform a more general study of the direct triangular decomposition $A = LU$ of a matrix A, where the diagonal elements of L are not necessarily unity. The induction formula thus becomes

$$\begin{aligned} u_{ij} &= \frac{1}{l_{ii}} \left(a_{ij} - \sum_{k=1}^{i-1} l_{ik} u_{kj} \right), \qquad i \leqslant j \\ l_{ij} &= \frac{1}{u_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} u_{kj} \right), \qquad i \geqslant j \end{aligned} \tag{2.21}$$

For each row and column, there is a degree of freedom in the choice of the diagonal elements of L and U, given by

$$l_{ii} u_{ii} = a_{ii} - \sum_{k=1}^{i-1} l_{ik} u_{ki}$$

If the product $l_{ii}u_{ii}$ is zero, then the factorization fails, unless all of the brackets vanish in (**??**) for the submatrices later on. When A is non-singular, this may happen if there need row or column interchanges in Gaussian elimination. If A is non-singular, then the use of maximal column pivots results in the sequence $(i_1, 1), (i_2, 2), \ldots, (i_n, n)$ as pivotal elements. Hence, the Gaussian elimination process shows that the triangular decomposition

$$LU = P^T A$$

for some row-interchange permutation matrix P. In the procedure, if we need to interchange row, we can simply interchange the corresponding rows of L and A, and then apply the formulae in (**??**) to the resulting matrix. At each step, simply calculate the entries of (l_{in}) for some n and find the maximum among them, and then interchange the corresponding rows of L and A.

Theorem 2.3.1: Direct LU Factorization

If A is non-singular, a triangular decomposition $LU = A$ may not exist. But a permutation of the rows of A always exists such that $B = P^T A = LU$ is a triangular decomposition.
P can also be chosen so that

$$|l_{kk}| \geqslant |l_{ik}| \qquad i > k, \quad k = 1, 2, \ldots, n-1$$

Example: **Direct LU Factorization with Partial Pivoting**

To illustrate the procedure with the stability condition $|l_{kk}| \geqslant |l_{ik}|$, consider $A = \left(\begin{smallmatrix} 1 & 1 & 1 \\ 2 & 2 & 5 \\ 4 & 6 & 8 \end{smallmatrix}\right)$ using the convention $u_{ii} = 1$. At step $k = 1$, we calculate the column candidates for L from the current first column of A, $(|1|, |2|, |4|)$. Since 4 is maximal, we interchange row 1 and row 3, resulting in the pivot $l_{11} = 4$ and multipliers $l_{21} = 2, l_{31} = 1$; the first row of U is then $u_{12} = 6/4 = 1.5$ and $u_{13} = 8/4 = 2$. At step $k = 2$, we compute candidates for the remaining L entries: for row 2, $a_{22} - l_{21}u_{12} = 2 - (2)(1.5) = -1$; for row 3, $a_{32} - l_{31}u_{12} = 1 - (1)(1.5) = -0.5$. Since $|-1| > |-0.5|$, no swap is needed. We set $l_{22} = -1, l_{32} = -0.5$ and solve for $u_{23} = (a_{23} - l_{21}u_{13})/l_{22} = (5 - 4)/(-1) = -1$. Finally, $l_{33} = a_{33} - (l_{31}u_{13} + l_{32}u_{23}) = 1 - (2 + 0.5) = -1.5$. The resulting factorization $LU = P^T A$ is:

$$L = \begin{pmatrix} 4 & 0 & 0 \\ 2 & -1 & 0 \\ 1 & -0.5 & -1.5 \end{pmatrix}, \quad U = \begin{pmatrix} 1 & 1.5 & 2 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix}, \quad P^T A = \begin{pmatrix} 4 & 6 & 8 \\ 2 & 2 & 5 \\ 1 & 1 & 1 \end{pmatrix}$$

We can also append the inhomogeneous vector f to the right of A and apply the above procedure to the resulting augmented matrix as usual.

2.3.1 Cholesky Factorization

The direct factorization method can be further simplified if A is positive definite.

Theorem 2.3.2: Cholesky Factorization

If A is a positive definite matrix, then there exists a unique lower triangular matrix L with positive diagonal elements such that

$$A = LL^T$$

A count of the arithmetic operations can be made if we remark that only the elements in the lower triangular part of A are required. If we count the square root operations separately, we have

$$\frac{1}{6}n^3 + n^2 - \frac{1}{6}n \text{ ops.} + n \text{ square roots} \qquad \text{for Cholesky factorization}$$

In addition, to find x we must solve a triangular system which requires $\frac{1}{2}n^2 + \frac{1}{2}n$ operations. To apply our previous error analysis, we deduce from

$$a_{ii} = \sum_{k=1}^{i} l_{ik}^2$$

the bounds

$$|l_{ik}|^2 \leqslant a_{ii} \leqslant a = \max_{i,j} |a_{ij}|$$

For single precision square root and $G(n) = 1$, we have

Theorem 2.3.3: Error Estimate for Cholesky Factorization

Let A be a positive definite matrix and employ the Cholesky factorization method to solve $Ax = f$. Let t be sufficiently large then we have

$$\|\delta A\|_\infty \leqslant a(2n^2 + 3n^3)10^{1-t} \tag{2.22}$$

Under the hypotheses of theorem 2.1.4, where inner products are accumulated exactly, prior to a final rounding, then for t sufficiently large, we have

$$\|\delta A\|_\infty = \mathcal{O}(n^2 a 10^{-t}) \tag{2.23}$$

2.3.2 Tridiagonal or Jacobi Matrices

A coefficient matrix which frequently occurs is the tridiagonal or Jacobi form, in which $a_{ij} = 0$ for $|i - j| > 1$. That is,

$$A = \begin{pmatrix} a_1 & c_1 & 0 & \cdots & 0 \\ b_2 & a_2 & c_2 & \cdots & 0 \\ 0 & b_3 & a_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & a_n \end{pmatrix} \tag{2.24}$$

Assume this matrix can be factored in the bidiagonal form

$$A = LU \equiv \begin{pmatrix} \alpha_1 & & & & \\ b_2 & \alpha_2 & & & \\ & b_3 & \alpha_3 & & \\ & & \ddots & \ddots & \\ & & & b_n & \alpha_n \end{pmatrix} \begin{pmatrix} 1 & \gamma_1 & & & \\ & 1 & \gamma_2 & & \\ & & 1 & \ddots & \\ & & & \ddots & \gamma_{n-1} \\ & & & & 1 \end{pmatrix}. \tag{2.25}$$

Them we find

- $\alpha_1 = a_1, \gamma_1 = c_1/\alpha_1$;
- $\alpha_i = a_i - b_i\gamma_{i-1}, i = 2, 3, \ldots, n$;
- $\gamma_i = c_i/\alpha_i, i = 2, 3, \ldots, n-1$.

Thus, if none of the α_i vanish, the factorization is accomplished by evaluating the recursions.

The intermediate solution g is given by

$$g_1 = f_1/\alpha_1, \qquad g_i = (f_i - b_i g_{i-1})/\alpha_i, \quad i = 2, 3, \ldots, n$$

and the final solution is given by

$$x_n = g_n, \qquad x_i = g_i - \gamma_i x_{i+1}, \quad i = n-1, n-2, \ldots, 1$$

In many of the applications of this procedure, the elements of A satisfies:

- $|a_1| > |c_1| > 0$;

- $|a_i| \geqslant |b_i| + |c_i|, b_i c_i \neq 0$ for $i = 2, 3, \ldots, n-1$;
- $|a_n| > |b_n| > 0$.

In such cases, the quantities α_i and γ_i are nicely bounded and in fact A is non-singular:

Theorem 2.3.4: Tridiagonal Matrices

If the elements of A satisfy the above conditions, then $\det A \neq 0$ and the elements of L and U are bounded by

- $|\gamma_i| < 1$;
- $|a_i| - |b_i| < |\alpha_i| < |a_i| + |b_i|$

Proof. We have $|\gamma_1| = |c_1|/|\alpha_1| < 1$. And

$$\gamma_i = \frac{c_i}{a_i - b_i\gamma_{i-1}}, \qquad |\gamma_i| \leqslant \frac{|c_i|}{|a_i| - |b_i||\gamma_{i-1}|} < \frac{|c_i|}{|a_i| - |b_i|} < 1.$$

Also, we have $\alpha_i = a_i - b_i\gamma_{i-1}$, thus it is easily proved.

The non-singularity of A follows from

$$\det A = \prod_{i=1}^{n} \alpha_i \neq 0$$

or from the Gershgorin circle theorem. □

The operational count for this procedure is somewhat striking. A total or

$$(3n-2)m + 2n - 2 \qquad \text{ops. for } m \text{ tridiagonal systems}$$

Consequently, the inverse can be obtained, although it should never be used in such circumstances to solve the system, in not more than

$$3n^2 - 2 \qquad \text{ops. for the inverse of a tridiagonal matrix}$$

The low operational counts are due to the fact that the zero elements of A have been accounted for in performing the calculations.

It should be observed that the factorization computed is not unique, for instance, we could try the form

$$A = LU \equiv \begin{pmatrix} 1 & & & & \\ \beta_2 & 1 & & & \\ & \beta_3 & 1 & & \\ & & \ddots & \ddots & \\ & & & \beta_n & 1 \end{pmatrix} \begin{pmatrix} \alpha_1 & c_1 & & & \\ & \alpha_2 & c_2 & & \\ & & \alpha_3 & \ddots & \\ & & & \ddots & c_{n-1} \\ & & & & \alpha_n \end{pmatrix}$$

There is a generalization called the **band matrix**:

Definition 2.3.1: Band Matrix

A matrix A is called a band matrix of width (b, a) if $a_{ij} = 0$ for $j - i \geqslant a$ and $i - j \geqslant b$.

2.3.3 Block-Tridiagonal Matrices

Another form which is encountered frequently, especially in the numerical solution of partial differential equations and integral equations, is the so-called block-tridiagonal matrix:

$$A = \begin{pmatrix} A_1 & C_1 & 0 & \cdots & 0 \\ B_2 & A_2 & C_2 & \cdots & 0 \\ 0 & B_3 & A_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & A_n \end{pmatrix} \tag{2.26}$$

Here, each of the A_i represents a square matrix, of order m_i, and the B_i and C_i are rectangular matrices.

This system may be solved by a procedure formally analogous to the previous factorization of a Jacobi matrix. Thus, let the system be $Ax = f$, where $x = (x^{(1)}, x^{(2)}, \ldots, x^{(n)})^T$ and $f = (f^{(1)}, f^{(2)}, \ldots, f^{(n)})^T$ are partitioned conformably with A.

Remark:
Thus, the method to be described is a special case of more general methods known as group- or block-elimination.

We seek a factorization of the form

$$A = LU \equiv \begin{pmatrix} L_1 & & & & \\ B_2 & L_2 & & & \\ & B_3 & L_3 & & \\ & & \ddots & \ddots & \\ & & & B_n & L_n \end{pmatrix} \begin{pmatrix} I_1 & \Gamma_1 & & & \\ & I_2 & \Gamma_2 & & \\ & & I_3 & \ddots & \\ & & & \ddots & \Gamma_{n-1} \\ & & & & I_n \end{pmatrix} \tag{2.27}$$

where I_i is the identity matrix of order m_i. Then we have

- $L_1 = A_1, \Gamma_1 = L_1^{-1} C_1$;
- $L_i = A_i - B_i \Gamma_{i-1}, i = 2, 3, \ldots, n$;
- $\Gamma_i = L_i^{-1} C_i, i = 2, 3, \ldots, n-1$.

The system now is equivalent to

$$Ly = f, \qquad Ux = y$$

where $y = (y^{(1)}, y^{(2)}, \ldots, y^{(n)})^T$ is an intermediate solution vector:

$$y^{(1)} = L_1^{-1} f^{(1)}, \qquad y^{(i)} = L_i^{-1}(f^{(i)} - B_i y^{(i-1)}), i = 2, 3, \ldots, n$$

$$x^{(n)} = y^{(n)}, \qquad x^{(i)} = y^{(i)} - \Gamma_i x^{(i+1)}, i = n-1, n-2, \ldots, 1$$

This method requires (or rather seems to require) the inversion of the n matrices A_i. To estimate the total number of operations used, we consider the cases where all $m_i = m$. Then with Gaussian elimination to obtain the inverses, we require

$$nm^3 \qquad \text{ops. for the inverses of } A_i$$

The product of two square matrices of order m requires m^3 operations, thus we have

$$2(n-1)m^3 \qquad \text{ops. for the products } A_i^{-1}C_i \text{ and } B_i\Gamma_{i-1}$$

Thus, the evaluation involves not more than

$$(3n-2)m^3 \qquad \text{ops. for the factorization}$$

To solve the system, we require

$$(3n-2)(m^3+m^2) \qquad \text{ops. for the solution}$$

Notice that this number is much superior to estimates of the form $\frac{1}{3}n^3m^3$ which would be obtained if the system were solved by Gaussian elimination without taking advantage of the block-tridiagonal structure.

The count is then reduced from the order of $3nm^3$ operations to the order of $\frac{5}{3}nm^3$ operations when we do not compute inverses explicitly, but directly solve the systems whenever we have one.

The justification of the block-factorization method is given in

Theorem 2.3.5: Block LU Factorization

If the leading diagonal blocks A_i are non-singular, then the block-tridiagonal matrix A can be factored as $A = LU$.

Proof. SORRY. □

2.4 Iterative Methods

The previous direct methods for solving general systems of order n require about $\frac{1}{3}n^3$ operations. In addition, it has been indicated that, in practical computations with these methods, the errors which are necessarily introduced through rounding may become quite large for large n.

Now we consider iterative methods in which an approximate solution is sought by using fewer operations per iteration. In general, these may be described as methods which proceed from some initial "guess" $x^{(0)}$ and define a sequence of successive approximations $x^{(1)}, x^{(2)}, \ldots$ which (hopefully) converges to the exact solution x of $Ax = f$. If the convergence is sufficiently rapid, the procedure may be terminated at an early stage in the sequence and will yield a good approximation.

Remark:
One of the intrinsic advantages of such methods is the fact that errors, due to roundoff or even blunders, may be damped out as the procedure continues. In fact, special iterative methods are frequently used to improve "solutions" obtained by direct methods.

A large class of iterative methods may be defined as follows: Let the system to be solved be $Ax = f$ where A is a non-singular matrix. We can write

$$A = N - P \tag{2.28}$$

Then the system is equivalent to

$$Nx = Px + f \tag{2.29}$$

tarting with some arbitrary vector $x^{(0)}$, we can define the sequence of approximations by

$$Nx^{(\nu)} = Px^{(\nu-1)} + f, \qquad \nu = 1, 2, \ldots \tag{2.30}$$

Thus we need the N in (**??**) to be non-singular:

$$\det N \neq 0$$

it is also clear that N should be chosen so that

$$Ny = z$$

can be easily solved for y when z is given.

If greater accuracy is desired, it would be better to calculate with (**??**) in the equivalent form:

$$N(x^{(\nu)} - x^{(\nu-1)}) = f - Ax^{(\nu-1)}$$

to get the actual error.

The convergence of the sequence $\{x^{(\nu)}\}$ to the solution x is studied by introducing

$$M = N^{-1}P, \qquad e^{(\nu)} = x^{(\nu)} - x, \qquad \nu = 0, 1, 2, \ldots \tag{2.31}$$

Then we have

$$e^{(\nu)} = Me^{(\nu-1)} = M^{\nu}e^{(0)}, \qquad \nu = 1, 2, \ldots$$

Thus, it is clear that a sufficient condition for convergence is that $\lim_{\nu\to\infty} M^{\nu} = 0$*, and this is also necessary if the method is to converge for all initial guesses* $x^{(0)}$. A matrix M that satisfies this condition is called a **convergent matrix**.

Theorem 2.4.1: Convergent Matrix

A matrix M is convergent if and only if its spectral radius $\rho(M)$ is less than unity, that is, all eigenvalues $|\lambda| < 1$.
This is also equivalent to: for any matrix norm $\|\cdot\|$, we have $\|M\| < 1$.

So, if we find a matrix norm such that $\|M\| < 1$, then the method will converge.

Definition 2.4.1: Rate of Convergence

The rate of convergence of the method is defined as:

$$R = -\log \rho(M) \tag{2.32}$$

As we have

$$\rho(M) = \sup_{\|\cdot\|} \|M\|$$

for all possible matrix norms. Then for a given $\epsilon > 0$, there is a norm that

$$\|e^{(\nu)}\| \leqslant \|M\|^{\nu}\|e^{(0)}\| \leqslant (\rho(M) + \epsilon)^{\nu}\|e^{(0)}\|$$

if $e^{(0)}$ is an eigenvector corresponding to the largest eigenvalue λ of M, then $\|e^{(\nu)}\| = (\rho(M))^{\nu}\|e^{(0)}\|$. Let it be required to reduce the amplitude of the error by a factor of at least 10^{-m}, then we see

that, in some norm, the error amplitude is reduced by a factor close to $\rho(M)^\nu$. The number of iterations required is the least value of ν such that $\rho(M)^\nu \leqslant 10^{-m}$, that is,

$$\nu \geqslant \frac{m}{-\log \rho(M)} = \frac{m}{R} \tag{2.33}$$

Thus, the number of iterations required to reduce the initial error by the factor of 10^{-m} is inversely proportional to the rate of convergence R.

To estimate the error from the iteration stepping $\delta x^{(\nu)} = x^{(\nu+1)} - x^{(\nu)}$, we have

$$\delta x^{(\nu)} = x^{(\nu+1)} - x^{(\nu)} = Mx^{(\nu)} + N^{-1}f - x^{(\nu)} = Mx^{(\nu)} + N^{-1}Ax - x^{(\nu)} = -(I - M)e^{(\nu)}$$

Thus we have

$$\|e^{(\nu)}\| \leqslant \frac{1}{1 - \|M\|}\|\delta x^{(\nu)}\|.$$

2.4.1 Jacobi or Simultaneous Iteration

A special case (attributed to Jacobi) of the previous general theory is

$$N = (a_{ii}\delta_{ij}), \qquad P = N - A \tag{2.34}$$

The iteration is then given by

$$x_i^{(\nu)} = \frac{1}{a_{ii}}\left(f_i - \sum_{j\neq i} a_{ij}x_j^{(\nu-1)}\right), \qquad i = 1, 2, \ldots, n \tag{2.35}$$

Thus, this procedure may be employed provided only that $a_{ii} \neq 0$ for $i = 1, 2, \ldots, n$.

However, for the convergence of these iterations, theorem **??** requires that all roots of $\det|\lambda I - N^{-1}P| = 0$ lie within the unit circle. Writing out,

$$\det|\lambda N - P| = \det\begin{pmatrix} \lambda a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & \lambda a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & \lambda a_{nn} \end{pmatrix} = 0$$

In general, the roots of such an equation, for large n, are not easily obtained and so we seek simpler sufficient conditions for convergence.

We have

$$M = N^{-1}P = \left(\delta_{ij} - \frac{a_{ij}}{a_{ii}}\right)$$

Then consider

$$\|M\|_\infty = \max_i \sum_{i\neq j, j=1}^{n} \left|\frac{a_{ij}}{a_{ii}}\right|, \qquad \|M\|_1 = \max_j \sum_{i\neq j, i=1}^{n} \left|\frac{a_{ij}}{a_{ii}}\right|$$

if either of these is less than unity, then the method will converge. The rate of convergence is then given by

$$R = -\log \rho(M) \geqslant -\log \min\{\|M\|_\infty, \|M\|_1\}$$

The operational count for the Jacobi iteration is simply obtained from

$$n^2 \qquad \text{ops. for each iteration}$$

if these iterations converge they require a total of about

$$\frac{mn^2}{R} \qquad \text{ops. to reduce the error by a factor of } 10^{-m}$$

We see that if such an iterative method is to be at least as efficient as the direct elimination method it should have a rate of convergence and required accuracy factor 10^{-m} such that

$$\frac{m}{R} \leqslant \frac{1}{3}n$$

2.4.2 Gauss-Seidel or Successive Iteration

It is clear that in the ordinary Jacobi iterations some components of $x^{(\nu)}$ are known, but not used, while computing the remaining components. The Gauss-Seidel method is a modification of the Jacobi method in which all of the latest known components are used.

Remark:
The term "successive iteration" is frequently applied to this method refers to the fact that "new" components are successively used as they are obtained. (In contrast, the previous scheme was called "simultaneous iteration" because all components of $x^{(\nu)}$ are computed simultaneously from the previous iterate $x^{(\nu-1)}$.)

The obvious modification suggests

$$x_i^{(\nu)} = \frac{1}{a_{ii}}\left(f_i - \sum_{j<i} a_{ij}x_j^{(\nu)} - \sum_{j>i} a_{ij}x_j^{(\nu-1)}\right), \qquad i = 1, 2, \ldots, n \tag{2.36}$$

The splitting is now given by

$$N = \begin{pmatrix} a_{11} & 0 & \cdots & 0 \\ a_{21} & a_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix}, \qquad P = N - A = \begin{pmatrix} 0 & a_{12} & \cdots & a_{1n} \\ 0 & 0 & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{pmatrix} \tag{2.37}$$

Since N is triangular, $\det N = \prod_{i=1}^{n} a_{ii} \neq 0$ requires only that $a_{ii} \neq 0$ for $i = 1, 2, \ldots, n$. The characteristic equation, whose roots must be in absolute value less than unity for convergence, is now of the form

$$\det \begin{pmatrix} \lambda a_{11} & a_{12} & \cdots & a_{1n} \\ \lambda a_{21} & \lambda a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \lambda a_{n1} & \lambda a_{n2} & \cdots & \lambda a_{nn} \end{pmatrix} = 0$$

The roots of this equation are just as difficult to find, so as computing $\|M\|_\infty$ and $\|M\|_1$ for the Jacobi method.

However, a simple sufficient condition for convergence of the Gauss-Seidel method can be obtained. The error vectors must satisfy

$$e_i^{(\nu)} = -\sum_{j<i} \frac{a_{ij}}{a_{ii}} e_j^{(\nu)} - \sum_{j>i} \frac{a_{ij}}{a_{ii}} e_j^{(\nu-1)}, \qquad i = 1, 2, \ldots, n$$

Remark:
This form is simply get if we view error vectors as "homogeneous" solutions of the system $Ax = 0$.

The result to be proved may now be stated as

Lemma 2.4.1: Sufficient Condition for Convergence of Gauss-Seidel Method

Let $e^{(0)}$ be an arbitrary initial error vector. Take maximum norm $\|e^{(\nu)}\|_\infty = \max_i |e_i^{(\nu)}|$, and if A is strictly diagonally dominant, that is,

$$r_i = \sum_{j \neq i} \left| \frac{a_{ij}}{a_{ii}} \right|, \qquad r = \max_i r_i < 1 \tag{2.38}$$

Then we have

$$\|e^{(\nu)}\|_\infty \leqslant r^\nu \|e^{(0)}\|_\infty, \tag{2.39}$$

and it converges to zero as $\nu \to \infty$.

Proof. We shall only need to show that $\|e^{(\nu)}\|_\infty \leqslant r\|e^{(\nu-1)}\|_\infty$ for $\nu = 1, 2, \ldots$. We have

$$|e_1^{(\nu)}| \leqslant \sum_{j>1} \left| \frac{a_{1j}}{a_{11}} \right| |e_j^{(\nu-1)}| \leqslant \|e^{(\nu-1)}\|_\infty \sum_{j>1} \left| \frac{a_{1j}}{a_{11}} \right| = r_1 \|e^{(\nu-1)}\|_\infty \leqslant r\|e^{(\nu-1)}\|_\infty$$

Now assume $|e_k^{(\nu)}| \leqslant r\|e^{(\nu-1)}\|_\infty$ for $k = 1, 2, \ldots, i-1$. Then

$$|e_i^{(\nu)}| \leqslant \sum_{j<i} \left| \frac{a_{ij}}{a_{ii}} \right| |e_j^{(\nu)}| + \sum_{j>i} \left| \frac{a_{ij}}{a_{ii}} \right| |e_j^{(\nu-1)}| \leqslant \|e^{(\nu-1)}\|_\infty \left(r \sum_{j<i} \left| \frac{a_{ij}}{a_{ii}} \right| + \sum_{j>i} \left| \frac{a_{ij}}{a_{ii}} \right| \right)$$

Recalling that $r < 1$, we have

$$|e_i^{(\nu)}| \leqslant \|e^{(\nu-1)}\|_\infty \sum_{j \neq i} \left| \frac{a_{ij}}{a_{ii}} \right| = r_i \|e^{(\nu-1)}\|_\infty \leqslant r\|e^{(\nu-1)}\|_\infty$$

Thus, the induction argument is complete and since the above inequality is valid. □

The convergence test of this lemma is easily applied and is formally identical to that of the Jacobi method. However, it is not generally true that if the Gauss-Seidel method converges then the Jacobi method will converge, nor is the converse generally true.

2.4.3 Method of Residue Correction

This iterative scheme improves upon the accuracy of the approximate solution of $Ax = f$, obtained for example by the Gaussian elimination method, by using the approximate numerical triangular factorization of A.

Consider the triangular factorization A performed with t digits, producing $\mathcal{L}, \mathcal{U}$ and $x^{(0)}$. Now define

$$N = \mathcal{LU}, \qquad P = N - A, \qquad r^{(0)} = f - Ax^{(0)}. \tag{2.40}$$

Observe that N is easily invertible, or $\mathcal{LU}y = z$ can be easily solved for y when z is given, by mere $n(n+1)$ operations (if $\mathcal{L}$ has unit diagonal elements, the count is n^2).

The iteration is convergent if $M = N^{-1}P = I - N^{-1}A$ is a convergent matrix. We have

$$\|M\| = \|I - N^{-1}A\| < 1$$

for some matrix norm.

Lemma 2.4.2: Sufficient Condition for Convergence

If $\|P\| \cdot \|A^{-1}\| < 1/2$, then the iterative method converges.

Proof. We have

$$\|M\| = \|I - N^{-1}A\| = \|N^{-1}P\| \leqslant \|N^{-1}\| \cdot \|P\|$$

Now, we have

$$N^{-1} = (A + P)^{-1} = (I + A^{-1}P)^{-1}A^{-1}$$

The Neumann series expansion of $(I + A^{-1}P)^{-1}$ is given by

$$(I + A^{-1}P)^{-1} = I - A^{-1}P + (A^{-1}P)^2 - (A^{-1}P)^3 + \cdots$$

if $\|A^{-1}P\| < 1$, then the above series converges and we have

$$\|(I + A^{-1}P)^{-1}\| \leqslant 1 + \|A^{-1}P\| + \|A^{-1}P\|^2 + \cdots = \frac{1}{1 - \|A^{-1}P\|}$$

The condition $\|A^{-1}P\| < 1/2$ implies that $\|(I + A^{-1}P)^{-1}\| < 2$. Thus, we have

$$\|M\| = \|N^{-1}P\| = \|(I + A^{-1}P)^{-1}A^{-1}P\| \leqslant \|(I + A^{-1}P)^{-1}\| \cdot \|A^{-1}P\| < 2 \cdot \frac{1}{2} = 1$$

The method of residue correction converges. □

Remark:
The principle is to write M in the form of $A^{-1}P$:

$$M = N^{-1}P = (I + A^{-1}P)^{-1}A^{-1}P$$

In practice, we do not directly compute

$$\mathcal{LU}x^{(\nu)} = (\mathcal{LU} - A)x^{(\nu-1)} + f.$$

If we introduce the *change* in the iterate and residue, we have

$$\delta x^{(\nu)} = x^{(\nu+1)} - x^{(\nu)}, \qquad r^{(\nu)} = f - Ax^{(\nu)} \tag{2.41}$$

Then we would get

$$\mathcal{LU}\delta x^{(\nu)} = r^{(\nu)} \tag{2.42}$$

and the computations are done with these equations.

For operational count, we need n^2 operations to evaluate $r^{(\nu)}$ so each iteration step requires $n(2n+1)$ operations, and only $2n^2$ operations if $\mathcal{L}$ has unit diagonal elements.

Also, in practical computation, the numerical solution of the equation would induce perturbations

$$\mathcal{L}_\nu \mathcal{U}_\nu \delta x^{(\nu)} = r^{(\nu)}, \qquad \mathcal{L}_\nu = \mathcal{L} + \Delta\mathcal{L}_\nu, \qquad \mathcal{U}_\nu = \mathcal{U} + \Delta\mathcal{U}_\nu$$

if we let

$$N_\nu = \mathcal{L}_\nu \mathcal{U}_\nu, \quad P_\nu = N_\nu - A, \quad M_\nu = N_\nu^{-1} P_\nu$$

Then we have

$$e^{(\nu)} = M_{\nu-1} e^{(\nu-1)} = M_{\nu-1} M_{\nu-2} \cdots M_0 e^{(0)}$$

If $\|M_\nu\| \leqslant q < 1$ for $\nu = 0, 1, 2, \ldots$, then we have $\|e^{(\nu)}\| \leqslant q^\nu \|e^{(0)}\|$ and the method converges for any initial guess $x^{(0)}$.

2.4.4 Positive Definite Systems

Many of the large order linear systems that arise in practice have real symmetric matrices which are positive definite. In such cases we can show that a quite general class of iteration methods converges. We state this result as

Theorem 2.4.2: Convergence of Iteration Methods for Positive Definite Systems

Let A be Hermitian of order n and N be any non-singular matrix for which

$$Q = N + N^* - A \tag{2.43}$$

is positive definite. Then the matrix

$$M = N^{-1}P = I - N^{-1}A$$

is convergent if and only if A is positive definite.

Proof. Take any eigenvalue λ and corresponding eigenvector u of M, then since N is non-singular, we have

$$Au = N(I - M)u = (1 - \lambda)Nu.$$

As A is positive definite, we have $Au \neq 0$ and thus $\lambda \neq 1$. By taking the complex inner product of each side, we have

$$\frac{1}{1-\lambda} = \frac{u^* N u}{u^* A u}, \qquad \frac{1}{\bar{\lambda}} = \frac{u^* N^* u}{u^* A u}$$

If we add these two equations we get

$$2\,\mathrm{Re}\left(\frac{1}{1-\lambda}\right) = \frac{u^*(N + N^*)u}{u^* A u},$$

if we write $\lambda = \alpha + i\beta$, then we have

$$\frac{2(1-\alpha)}{(1-\alpha)^2 + \beta^2} = 1 + \frac{u^* Q u}{u^* A u} > 1$$

Simplifying the above inequality we get

$$|\lambda|^2 = \alpha^2 + \beta^2 < 1. \tag{2.44}$$

The sufficiency is thus demonstrated.

To prove necessity, assume $\rho(M) < 1$. For any eigenvalue λ of M, we have $|\lambda|^2 < 1$. Using the previous identity:

$$\frac{u^*Qu}{u^*Au} = \frac{2(1-\alpha)}{(1-\alpha)^2+\beta^2} - 1 = \frac{1-(\alpha^2+\beta^2)}{(1-\alpha)^2+\beta^2} = \frac{1-|\lambda|^2}{|1-\lambda|^2}$$

Since $|\lambda| < 1$, the right-hand side is strictly positive. Because Q is given as positive definite, $u^*Qu > 0$. For the ratio to be positive, we must have $u^*Au > 0$ for every eigenvector u of M. Since A is Hermitian, and the eigenvectors of M can be shown to span the space in this context, it follows that A is positive definite. □

Remark:
Intuitively, we can view this as: if N is "larger" than $A/2$ in some sense, then $I - N^{-1}A$ is a contraction and thus convergent.

As an immediate corollary of this theorem, we have a result on the convergence of the Gauss-Seidel method for Hermitian matrices.

Corollary 2.4.1: Convergence of Gauss-Seidel Method for Hermitian Matrices

If A is Hermitian with positive diagonal elements, then the Gauss-Seidel method converges if and only if A is positive definite.

Proof. For the Gauss-Seidel method, we have

$$A = D + E + E^*, \qquad N = D + E, \qquad P = -E^*$$

where D is the diagonal part of A, E is the strictly lower triangular part of A, and E^* is the strictly upper triangular part of A. Thus we have $Q = N + N^* - A = D$ which is positive definite if the diagonal elements of A are positive. □

Similarly, we obtain a result on the convergence of the Jacobi iterations as a special case of

Corollary 2.4.2: Convergence of Jacobi Method for Hermitian Matrices

Let $D = D^*$ be non-singular and $D - (E + E^*)$ be positive definite, then $D^{-1}(E + E^*)$ is a convergent matrix if and only if $A = D + E + E^*$ is positive definite.

Proof. Use $N = D$. □

If D is diagonal this yields the convergence condition for the Jacobi method for matrix A.

2.4.5 Block Iteration Methods

There are other splittings of A which in many important cases yield rapidly convergent iterations. In particular, since tridiagonal and block-tridiagonal systems are easily solved, it is natural to consider iterations in which N has these forms.

More generally, if the elements are "close" to the diagonal of a matrix are large compared to the other elements, it is usually advantageous to include all of these large elements in N (assuming, of course, that the resulting systems which determine the iterates are still easily solved).

2.5 The Acceleration of Iteration Methods

Given any iteration procedure, for a specific system of equations, it may be possible to improve its rate of convergence by a simple device. Such modifications, which we call *acceleration*, are frequently termed *"extrapolation," "over-relaxation,"* or various other names depending upon the problem to which they are applied or perhaps upon the particular form of device which is used.

In any event, the general principle common to almost all acceleration procedures is the introduction of a splitting, which depends upon some real parameter α, in an appropriate way. The splitting is denoted as

$$A = N(\alpha) - P(\alpha), \qquad \det N(\alpha) \neq 0, \qquad M(\alpha) = N(\alpha)^{-1}P(\alpha) \tag{2.45}$$

The convergence of the iteration is achieved if all eigenvalues $\lambda_i(\alpha)$ of $M(\alpha)$ satisfy $|\lambda_i(\alpha)| < 1$ for $i = 1, 2, \ldots, n$, or

$$\rho(M(\alpha)) = \max_i |\lambda_i(\alpha)| < 1$$

the rate of convergence is given by

$$R(\alpha) = -\log \rho(M(\alpha))$$

The convergence of the iteration is "best" when $R(\alpha)$ is maximum, or equivalently

$$\rho(M(\alpha_*)) = \min_\alpha \rho(M(\alpha)) \tag{2.46}$$

The selection of an optimal value α_* is the problem of acceleration.

Some acceleration procedures that are commonly used are described as follows: Let some definite splitting be given by

$$A = N_0 - P_0, \qquad \det N_0 \neq 0, \qquad M_0 = N_0^{-1}P_0$$

the eiegenvalues of M_0 are λ_i for $i = 1, 2, \ldots, n$. Then we introduce the one-parameter family of splittings:

$$\begin{aligned} &N(\alpha) = (1+\alpha)N_0, \qquad P(\alpha) = (1+\alpha)N_0 - A = P_0 + \alpha N_0, \\ &M(\alpha) = N(\alpha)^{-1}P(\alpha) = \frac{1}{1+\alpha}M_0 + \frac{\alpha}{1+\alpha}I \end{aligned} \tag{2.47}$$

For $\det N(\alpha) \neq 0$, we only need $\alpha \neq -1$. The eigenvalues of $M(\alpha)$ are given by

$$\mu_i(\alpha) = \frac{\lambda_i + \alpha}{1+\alpha}, \qquad i = 1, 2, \ldots, n \tag{2.48}$$

and the eigenvectors of $M(\alpha)$ are the same as those of M_0.

In order to determine convergent schemes, we have

Theorem 2.5.1: Convergence of Accelerated Iteration Methods

Let N_0 and P_0 be given such that the eigenvalues λ_i of $M_0 = N_0^{-1}P_0$ are all real and satisfy

$$\lambda_1 \leqslant \lambda_2 \leqslant \cdots \leqslant \lambda_n < 1$$

Then the scheme defined by (**??**) is convergent for any α such that

$$\alpha > -\frac{1+\lambda_1}{2} > -1. \tag{2.49}$$

Furthermore, the largest rate of convergence for these schemes is obtained when

$$\alpha = \alpha_* = -\frac{\lambda_1+\lambda_n}{2}, \tag{2.50}$$

for which we have

$$\rho(M(\alpha_*)) = \min_\alpha \rho(M(\alpha)) = \min_\alpha \max_i |\mu_i(\alpha)| = \frac{\lambda_n - \lambda_1}{2-\lambda_1-\lambda_n} < 1. \tag{2.51}$$

Proof. The first part of the theorem is clear from (**??**), just solve the inequality $|\mu_i(\alpha)| < 1$ for $i = 1, 2, \ldots, n$. For the second part, we have

$$\rho(M(\alpha)) = \max\{|\mu_1(\alpha)|, |\mu_n(\alpha)|\}.$$

The function $\mu(\lambda) = \dfrac{\lambda+\alpha}{1+\alpha}$ is monotonic in λ. Therefore, the maximum absolute value occurs at the boundaries of the spectrum of M_0. The optimal α occurs when the two extreme eigenvalues are equal in magnitude but opposite in sign:

$$\mu_1(\alpha_*) = -\mu_n(\alpha_*) \implies \frac{\lambda_1+\alpha_*}{1+\alpha_*} = -\frac{\lambda_n+\alpha_*}{1+\alpha_*}.$$

Solving for α_*:

$$\lambda_1 + \alpha_* = -\lambda_n - \alpha_* \implies 2\alpha_* = -(\lambda_1+\lambda_n) \implies \alpha_* = -\frac{\lambda_1+\lambda_n}{2}.$$

Substituting α_* back into the expression for $\mu_n(\alpha)$:

$$\rho(M(\alpha_*)) = \frac{\lambda_n+\alpha_*}{1+\alpha_*} = \frac{\lambda_n-\lambda_1}{2-\lambda_1-\lambda_n}.$$

Since $\lambda_1 \leqslant \lambda_n < 1$, it follows that $\lambda_n - \lambda_1 \geqslant 0$ and the denominator $2-\lambda_1-\lambda_n > 0$. Furthermore, since $2-\lambda_1-\lambda_n > \lambda_n - \lambda_1$ (which simplifies to $2 > 2\lambda_n$, or $1 > \lambda_n$), we have $\rho(M(\alpha_*)) < 1$. □

Remark:
Another set can be found for $\lambda_i > 1$ for $i = 1, 2, \ldots, n$.

In the general case, the λ_i and hence $\mu_i(\alpha)$ can be complex. Then the schemes in (**??**) are convergent if $|\mu_i(\alpha)| < 1$ for $i = 1, 2, \ldots, n$. Here, the mapping

$$\mu = \frac{\lambda+\alpha}{1+\alpha}, \qquad \alpha \neq -1.$$

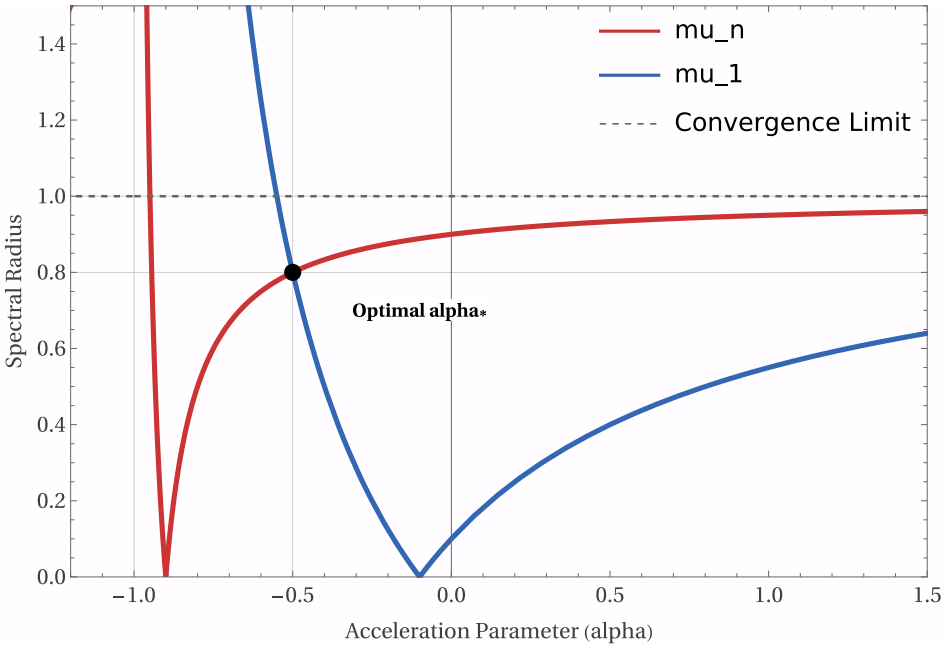

Figure 2.1: Optimized Alpha

is a well-known Ḿobius transformation of the complex plane.

We must note that a valid solution can be found for any circle in the λ plane that

- the center is on the real axis,
- passes through the point 1,
- all eigenvalues λ_i are interior to the circle.

If such a circle exists, then we call the coordinates of its center $(-\alpha, 0)$ and this value of a yields a convergent scheme. However, now it is not a simple matter to determine the best value of α.

2.5.1 Practical Acceleration Schemes

It is assumed that the basic scheme $A = N_0 - P_0$ can be efficiently computed:

$$N_0 x^{(\nu)} = P_0 x^{(\nu-1)} + f$$

We assume that such systems can be solved in an efficient manner. Now the iterates $x^{(\nu)}$ satisfy:

$$x^{(\nu)} = M_0 x^{(\nu-1)} + g, \qquad g = N_0^{-1} f$$

The accelerated (**??**) can be written as

$$y^{(\nu)} = M(\alpha) y^{(\nu-1)} + \frac{1}{1+\alpha} g = \frac{\alpha}{1+\alpha} y^{(\nu-1)} + \frac{1}{1+\alpha}(M_0 y^{(\nu-1)} + g). \tag{2.52}$$

If $\alpha = 0$, then the basic scheme is recovered. If $\alpha > 0$, then for any $a, b \in \mathbb{R}$ we have

$$\min\{a, b\} \leqslant \frac{\alpha a + b}{1+\alpha} \leqslant \max\{a, b\}$$

So in this case, the acceleration scheme yields a vector on the line segment joining the previous iterate and what would have been the next iterate of the basic scheme. The term "interpolation" is often applied to this case.

If $-1 < \alpha < 0$, then

$$\frac{\alpha}{1+\alpha}a + \frac{1}{1+\alpha}b \begin{cases} \leqslant b, & \text{if } b \leqslant a \\ \geqslant b, & \text{if } b \geqslant a \end{cases}$$

The scheme is now termed an "extrapolated iteration." Similarly, the remaining case $\alpha < -1$ is an "extrapolated iteration" in another direction.

To compute using the scheme (**??**), we let

$$N_0 z^{(\nu)} = P_0 y^{(\nu-1)} + f, \qquad y^{(\nu)} = \frac{\alpha}{1+\alpha} y^{(\nu-1)} + \frac{1}{1+\alpha} z^{(\nu)}$$

Thus, as in the basic scheme, the calculations only require the solution of systems of the form $N_0 z = w$.

In general, the eigenvalues λ_i of M_0 are not known, especially λ_1 and λ_n which me use to determine the optimal α. However, it may be possible to approximate the value α_* by some test calculations that are easily performed:

Since the rate of convergence is independent of the inhomogeneous term f, we can choose $f = 0$, and the system now has the form $Ay = 0$, having a unique solution $y = 0$. Take any non-zero initial guess $y^{(0)}$, and compute $y^{(\nu)}$ by

$$y^{(\nu)}(\alpha) = M(\alpha) y^{(\nu-1)} = M^{\nu}(\alpha) y^{(0)}$$

for some $\alpha = \alpha_1$, if α_1 yields a convergent scheme, then we compute until

$$\|y^{(\nu)}(\alpha_1)\| = \max_i |y_i^{(\nu)}(\alpha_1)| \leqslant 10^{-m}$$

where $m > 0$. This requires some minimum number of iterations which we may call $\nu(\alpha_1)$. Now repeat this for a sequence of values $\alpha_1, \alpha_2, \alpha_3, \ldots$ to obtain the corresponding sequence $\nu(\alpha_1), \nu(\alpha_2), \nu(\alpha_3), \ldots$. Choose the value α_k for which $\nu(\alpha_k)$ is minimum, and this value of α is an approximation to the optimal value α_*.

An alternative method is to compute $\|y^{(N)}(\alpha)\|$ for some fixed N and a sequence of values $\alpha_1, \alpha_2, \alpha_3, \ldots$, and choose the value α_k for which $\|y^{(N)}(\alpha_k)\|$ is minimum.

2.5.2 Generalizations of Acceleration Schemes

There are numerous generalizations of the acceleration method which in fact are more powerful. The simplest type of generalization proceeds from a single basic splitting but employs a sequence of parameters $\alpha_1, \alpha_2, \alpha_3, \ldots, \alpha_r$ to define a sequence of splittings

$$M(\alpha_i) = N(\alpha_i)^{-1} P(\alpha_i) = \frac{\alpha_i}{1+\alpha_i} I + \frac{1}{1+\alpha_i} M_0$$

The iterations are defined as follows, with $x^{(0)}$ arbitrary:

- $y^{(\nu,0)} = x^{(\nu-1)}$,
- $y^{(\nu,s)} = M(\alpha_s) y^{(\nu,s-1)} + N(\alpha_s)^{-1} f$ for $s = 1, 2, \ldots, r$,
- $x^{(\nu)} = y^{(\nu,r)}$.

Remark:
This can be seen as a "cycle" of r iterations of the basic acceleration scheme, with different values of α for each iteration.

again we can replace the need to compute inverse by solve an equation

$$N(\alpha_s)y^{(\nu,s)} = P(\alpha_s)y^{(\nu,s-1)} + f$$

With this notation, one iteration of this generalized acceleration scheme requires the same number of computations as r iterations in the ordinary acceleration scheme. The convergence of this method can be analyzed by means of the equivalent formulation:

$$\begin{aligned}
&x^{(\nu)} = M(\alpha_1, \alpha_2, \ldots, \alpha_r)x^{(\nu-1)} + g \\
&M(\alpha_1, \alpha_2, \ldots, \alpha_r) = M(\alpha_r)M(\alpha_{r-1}) \cdots M(\alpha_1) \\
&g = [N(\alpha_r)^{-1} + M(\alpha_r)N(\alpha_{r-1})^{-1} + \cdots + M(\alpha_r)M(\alpha_{r-1}) \cdots M(\alpha_2)N(\alpha_1)^{-1}]f
\end{aligned}$$

The eigenvalues of $M(\alpha_1, \alpha_2, \ldots, \alpha_r)$ are given by

$$\mu_i(\alpha_1, \alpha_2, \ldots, \alpha_r) = \prod_{s=1}^{r} \frac{\lambda_i + \alpha_s}{1 + \alpha_s}, \qquad i = 1, 2, \ldots, n \tag{2.53}$$

Now if we define the r-th degree polynomial

$$P_r(\lambda) = \prod_{s=1}^{r} \frac{\lambda + \alpha_s}{1 + \alpha_s}$$

then convergence is implied by $|P_r(\lambda_i)| < 1$ for $i = 1, 2, \ldots, n$. In particular, if all the eigenvalues of M_0 are real and satisfy $a \leqslant \lambda_i \leqslant b$, then convergence is implied by

$$|P_r(\lambda)| < 1 \text{ for all } a \leqslant \lambda \leqslant b. \tag{2.54}$$

In this case, the fastest convergence can be expected for that polynomial which has the smallest absolute magnitude in the indicated interval. Such problems are considered later.

Another type of generalization of the acceleration method is obtained by employing a sequence of different basic splittings, say

$$A = N_i - P_i, \qquad \det N_i \neq 0, \qquad M_i = N_i^{-1}P_i, \qquad i = 1, 2, \ldots, r$$

and their corresponding accelerated forms $N_i(\alpha_i)$, $P_i(\alpha_i)$, and $M_i(\alpha_i)$.

2.6 Matrix Inversion by High-Order Iteration

The previous methods of this chapter have been primarily concerned with solving linear systems. Of course, they can all be employed to determine the inverse of any given nonsingular matrix A. We consider now an iterative method for directly computing A^{-1}.

This method is a means for improving the accuracy of an approximate inverse, say R_0, obtained by some other process. However, in many cases the present method is feasible when the initial approximate inverse is assumed to have the simple form $R_0 = \omega I$ for some scalar ω. Because of

the large number of operations involved in matrix multiplication, these schemes are not generally used.

We define the error

$$E_0 = I - AR_0.$$

we define a sequence of approximate inverses by

- $R_\nu = R_{\nu-1}(I + E_{\nu-1} + E_{\nu-1}^2 + \cdots + E_{\nu-1}^{p-1})$ for $\nu = 1, 2, \ldots,$
- $E_\nu = I - AR_\nu$ for $\nu = 0, 1, 2, \ldots.$

here $p \geqslant 2$ is a fixed integer. We have

$$\begin{aligned} E_\nu &= I - AR_\nu \\ &= I - AR_{\nu-1}(I + E_{\nu-1} + E_{\nu-1}^2 + \cdots + E_{\nu-1}^{p-1}) \\ &= I - (I - E_{\nu-1})(I + E_{\nu-1} + E_{\nu-1}^2 + \cdots + E_{\nu-1}^{p-1}) \\ &= E_{\nu-1}^p. \end{aligned}$$

Thus, in one iteration, the error matrix is raised to the pth power and the method is consequently called the p-th order iteration method.

Remark:
Think of this backwards: Our goal is to find a iteration scheme that satisfies $E_\nu = E_{\nu-1}^p$. We can write

$$\begin{aligned} R_\nu &= A^{-1}(I - E_\nu) \\ &= A^{-1}(I - E_{\nu-1}^p) \\ &= A^{-1}(I - E_{\nu-1})(I + E_{\nu-1} + E_{\nu-1}^2 + \cdots + E_{\nu-1}^{p-1}) \\ &= R_{\nu-1}(I + E_{\nu-1} + E_{\nu-1}^2 + \cdots + E_{\nu-1}^{p-1}) \end{aligned}$$

Thus we have

$$E_\nu = E_0^{(p^\nu)} \tag{2.55}$$

Thus the iteration converges if E_0 is a convergent matrix. It true, then the error E_ν vanishes as $\rho(E_0)^{p^\nu}$ as $\nu \to \infty$.

We now pose the problem of determining the "best" value of p to be used for any convergent E_0. By the best value we mean that procedure which for the least amount of computation yields an approximate inverse of desired accuracy. Alternatively, the best scheme could be defined as the one for which a given amount of computation yields the most accurate inverse.

Adopting our usual convention we find from (2), since the product of two matrices requires n^3 operations, that ν iterations of a p-th order method requires

$$\nu p n^3 \qquad \text{ops. for } \nu \text{ iterations.}$$

If only K operations are to be permitted the number of iterations allowed is $\nu = K/(pn^3)$. Thus, the principal eigenvalue is reduced to

$$\rho^{p^\nu} = \rho^{p^{K/(pn^3)}} = \exp\left(p^{K/(pn^3)} \log \rho\right)$$

we find that the error is minimized when $p^{1/p}$ is a maximum, which is $e \approx 2.71828$. Also, we have

$$3^{1/3} \approx 1.44225 > 2^{1/2} \approx 1.41421.$$

So we see that the 3-rd order iteration method is the best of the integer-order methods.

In order to apply the procedure, we need to have an initial estimate R_0 such that $E_0 = I - AR_0$ is a convergent matrix. For a very important class of matrices such an estimate is easily found. This result is contained in

Theorem 2.6.1: Convergence of High-Order Iteration Method for Matrix Inversion

Let A have real eigenvalues in the interval

$$0 < m \leqslant \lambda_j \leqslant M, \qquad j = 1, 2, \ldots, n.$$

Then if $R_0(\omega) = \omega I$ and $E_0(\omega) = I - \omega A$, then $E_0(\omega)$ is a convergent matrix for all ω in

$$0 < \omega < \frac{2}{M}. \tag{2.56}$$

If $\rho(\omega)$ is the spectral radius of $E_0(\omega)$, then

$$\rho(\omega_*) = \min_{0<\omega<2/M} \rho(\omega) = \frac{M-m}{M+m} < 1, \qquad \omega_* = \frac{2}{M+m}. \tag{2.57}$$

Proof. Obvious as it is, also a restatement of theorem **??**. □